# AIと書く
# 予測と意図のあいだで

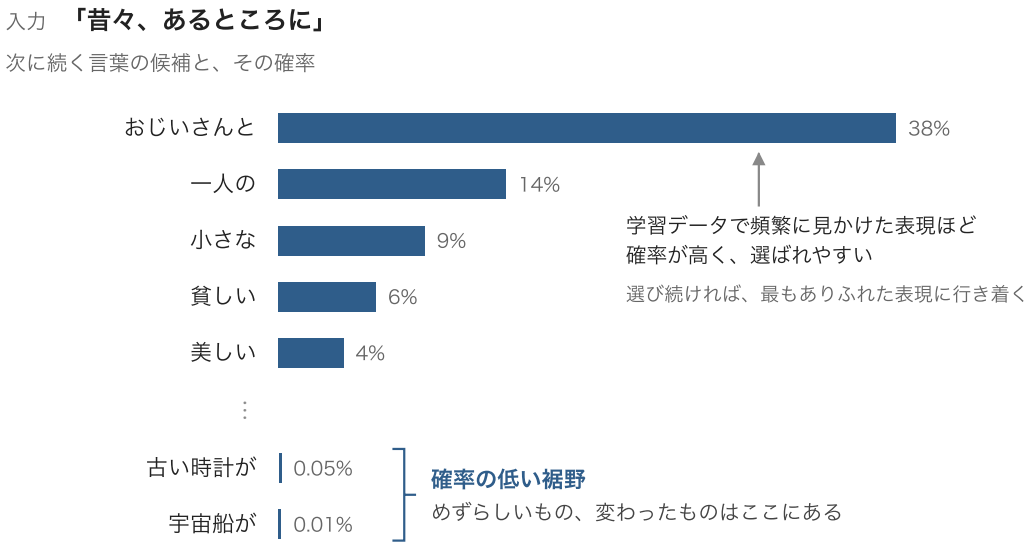
東京藝術大学芸術情報センター
田所 淳

## 今日の内容

- 前回の振り返り
  - 次の言葉の予測、ボタンの手前・奥・向こう、平均への引力
- ウォームアップ: 予測変換で書く
- VS Code と GitHub Copilot のセットアップ
- 補完の基本操作
- 4つの実験
  - 平均を観察する / 文脈を設計する / 平均から外れる / 疑う
- 次週までの課題
  - AIと共創するレポート「創作において人工知能とは…」

# 前回の振り返り

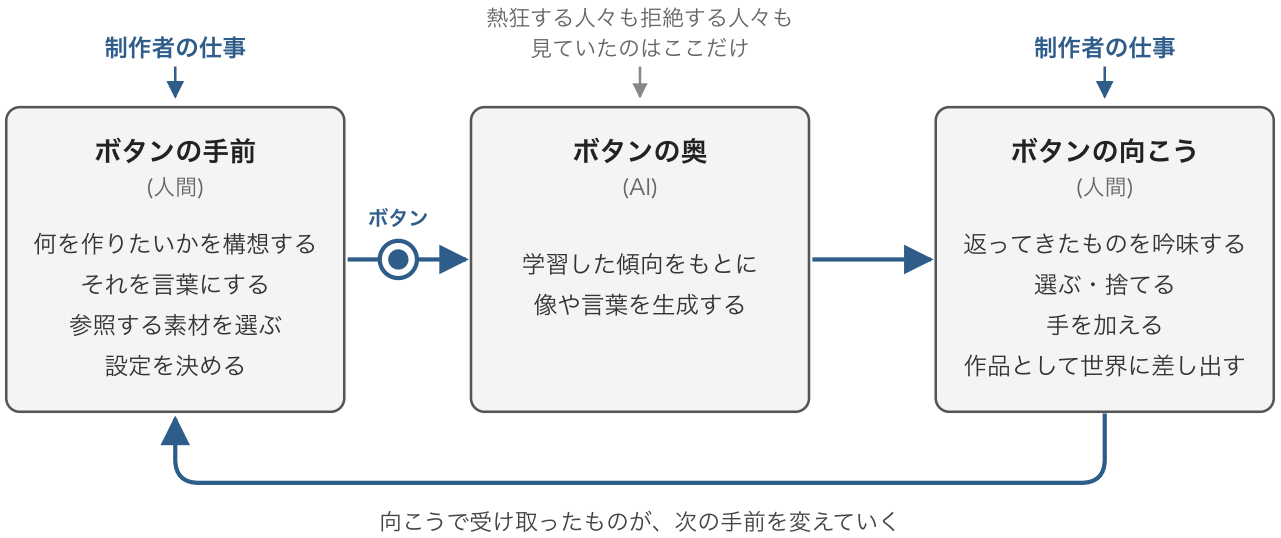
# 次の言葉を確率で予測する




※ 確率の値は説明のための模式的なもので、実際のモデルの出力ではありません。

- 言語モデルは、与えられた文章に続く **「次の言葉」を確率で予測** する
- 今日使う GitHub Copilot の補完も、このしくみで書きかけの文章の続きを予測する

# ボタンの手前・奥・向こう




- **手前**: 構想し、言葉にし、素材を選び、設定を決める
- **奥**: AIが像や言葉を生成する
- **向こう**: 吟味し、選び、手を加え、作品として世界に差し出す

## 平均への引力

- 言語モデルも画像生成AIも、**確率の高い方へ** と進んで生成する
  - 「夕日」→ 水平線に沈む太陽と橙色の空という **典型的な夕日**
- めずらしいもの、変わったものは **確率の低い裾野** にある
  - そこは、AIが最も苦手とする領域
- 人間の好みに合わせる調整で、出力は **なめらかで無難な方向** へ
- 平均から抜け出すには、人間の側の粘り強い働きかけが必要
  - Allen の624回のプロンプト、九段の20万字のプロンプト

## 今日の目標: 「使いこなす」とは?

- 第1回の Mentimeter より
  - 「AIを使いこなせるようになりたいけれど、使いこなせるとはどういう状態なのかも分からない」
- 今日のひとつの答え
  - **AIの提案を受け入れるか、書き換えるか、断るかを、自分で選べること**
- 補完は、この判断を何度も繰り返す練習にぴったりの環境
  - 前回の「手前・奥・向こう」の循環が、数秒ごとに回る

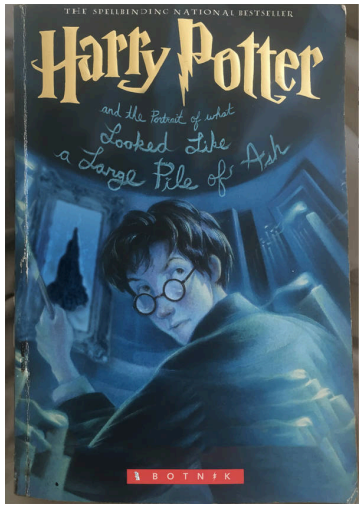
# ウォームアップ
# 予測変換で書く

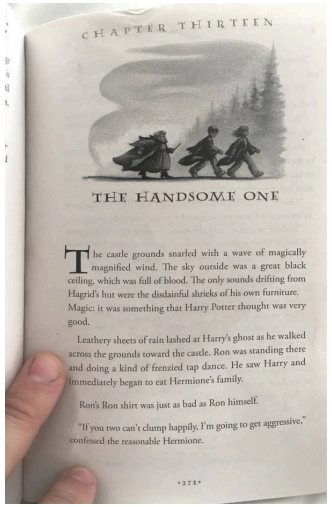
## スマホの予測変換だけで書いてみよう

- スマホのメモアプリで「**創作において人工知能とは、**」と入力
- そのあとは、キーボードに出る **予測変換の候補だけ** をタップして1分ほど続ける
- できあがった文章を、隣の人と見せ合う
- 予測変換も「次の言葉」を予測するしくみ
  - 入力の履歴も学習しているので、**自分の言葉の癖** が映る
  - Copilot に映るのは、膨大なテキストから学んだ **多くの人々の言葉の傾向**

> ⚠️ 見せたくない言葉が出てきたら、共有しなくてOK

# Botnik Studios『ハリー・ポッター』(2017)




- 全7巻を学習させた **予測キーボード** で書かれた、架空の続編の第13章
  - メンバーが候補の言葉を一語ずつ選んで文を作り、編集者が章にまとめた
- 候補を出したのは機械、**どれを選び、どう並べるかを決めたのは人間**
  - あえて奇妙な候補を選ぶことで、作品が生まれた

画像出典: Botnik Studios「Harry Potter」

# VS Code と GitHub Copilot のセットアップ

# AIとの共創の準備

- 文章を書きながら、その続きを AI がリアルタイムに提案する環境をつくる
  - エディタ: **Visual Studio Code** (VS Code)
  - AI: **GitHub Copilot** の **補完機能** (書きかけの文章の続きを提案)
- 準備の流れ (前回の事前準備で1・2を済ませた人は **3から**)
  i. VS Code をインストール
  ii. GitHub アカウントを作成
  iii. VS Code で AI 機能を有効化
  iv. GitHub でサインインして認可 (Copilot Free に自動登録)
  v. Markdown で補完を有効にする
  vi. テキスト補完を試す

> 💡 **拡張機能のインストールは不要**（VS Code 1.116 以降は本体に Copilot が標準搭載）

# 1. VS Code をインストール

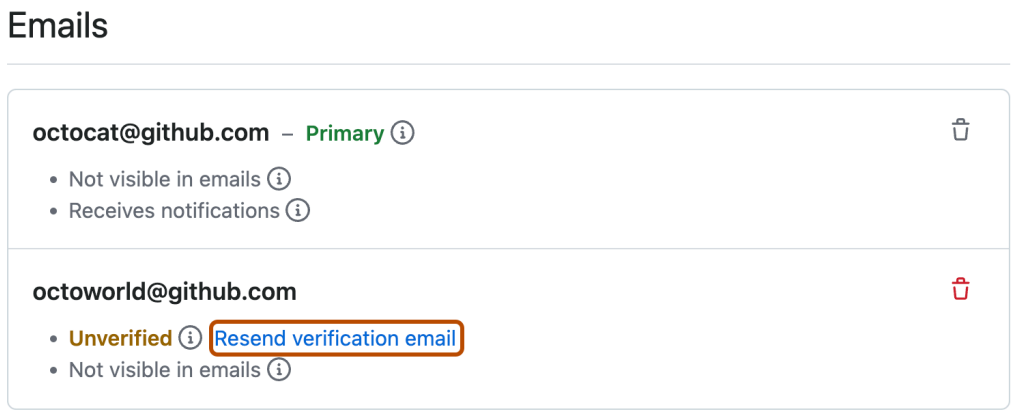
- 公式サイトからダウンロード: https://code.visualstudio.com/

| OS | インストール手順 |
| --- | --- |
| **Windows** | ダウンロードしたインストーラを実行（設定はデフォルトのままで OK） |
| **macOS** | zip を展開し、`Visual Studio Code.app` を「アプリケーション」フォルダへ移動 |

> ⚠️ **メニューの言語について**: 最初は英語で表示されます。このスライドでも英語UI名（Settings, Accounts など）で説明します。

# 2. GitHub アカウントを作成

- https://github.com/ を開いて **Sign up**（Google アカウントでも登録可能）
- メールアドレス・パスワード・ユーザー名・国/地域を入力し、パズルを解く
- メールに届く確認コードを入力して **メール認証を完了**
  - 認証状態は `Settings > Emails` で確認可能（Unverified なら再送できる）

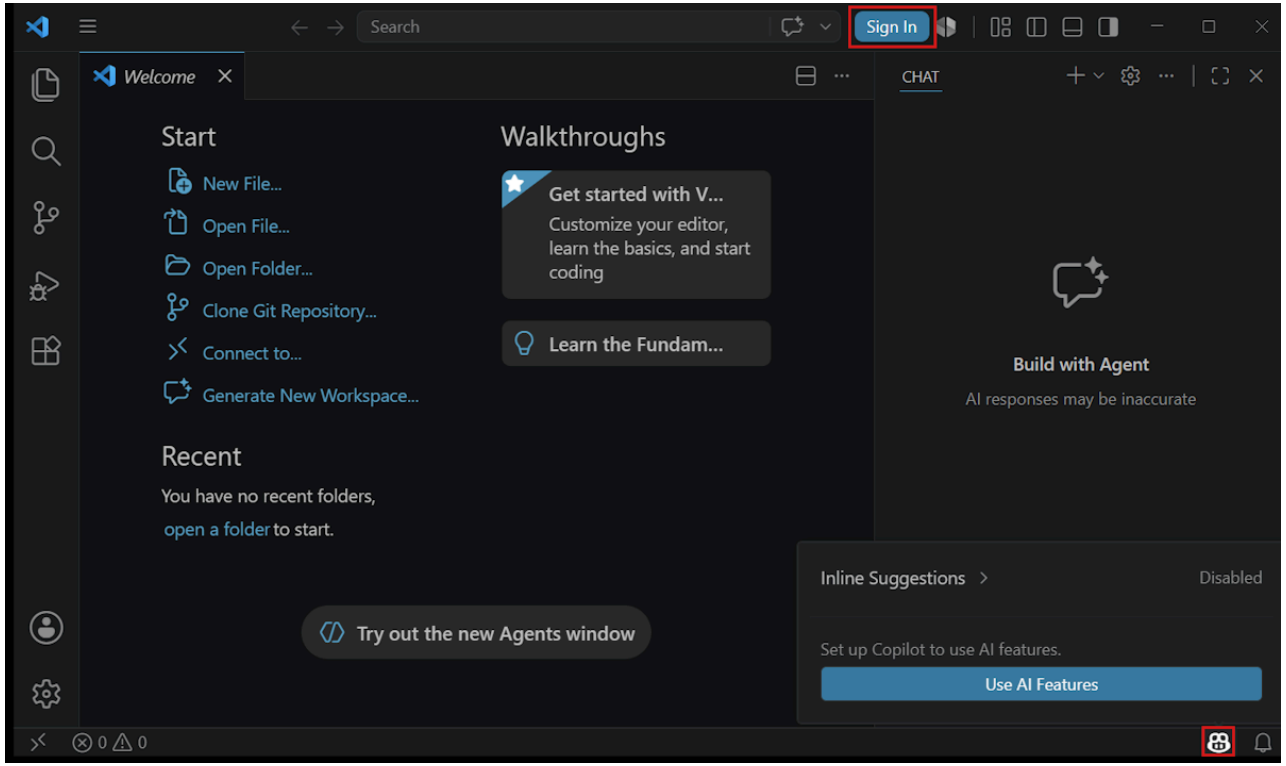



画像出典:GitHub Docs(CC BY 4.0)

# 3. VS Code で AI 機能を有効にする

VS Code を起動し、次のどちらかをクリック（図の赤枠参照）：

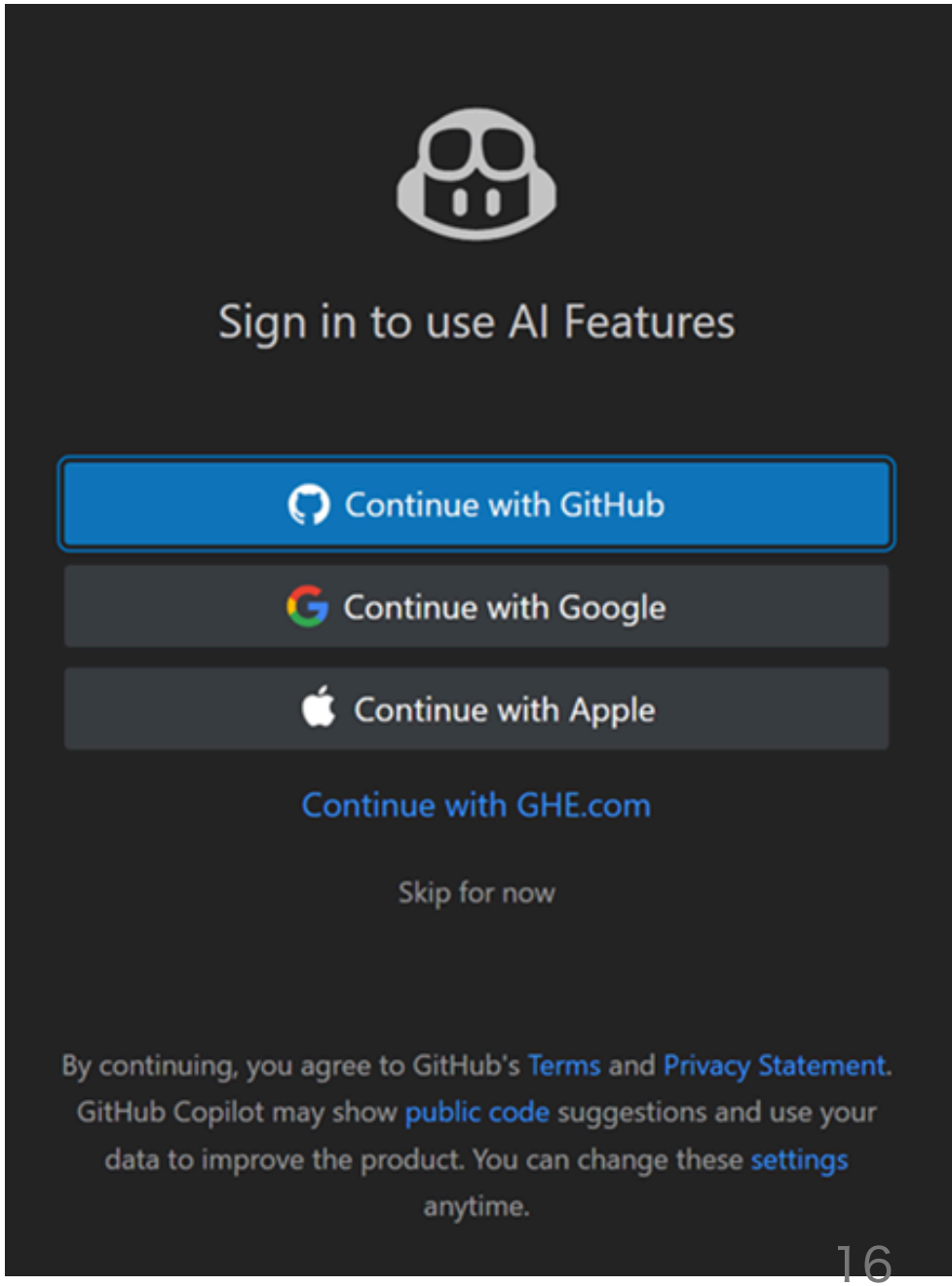
- **方法 A（推奨）**：タイトルバー右上の **Sign In**
- **方法 B**: 右下ステータスバーの Copilot アイコン → **Use AI Features**




画像出典:Visual Studio Code Docs(CC BY 3.0 US)

## 4-1. サインイン方法を選ぶ

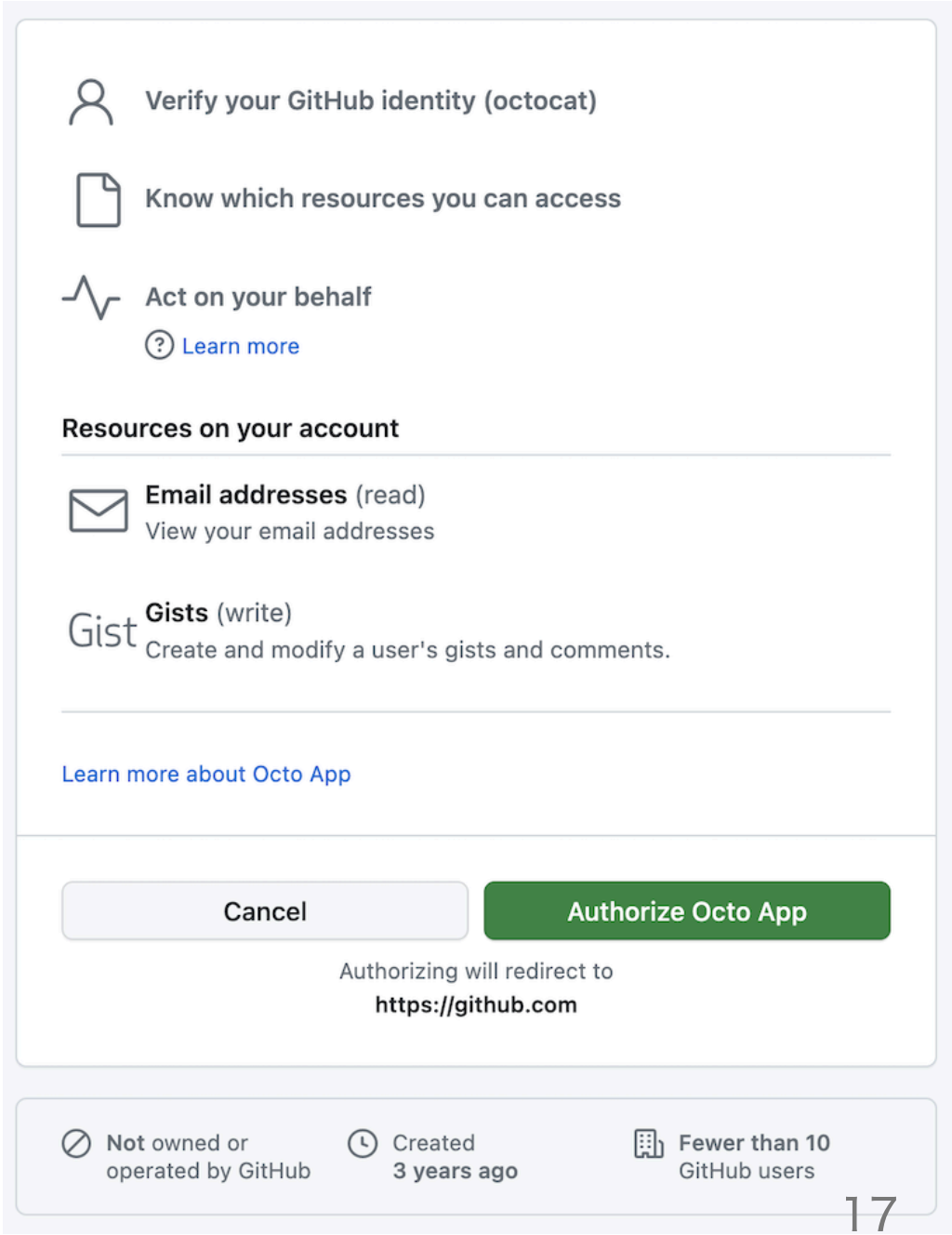
- **Continue with GitHub** を選ぶ
- 自動でブラウザが開くので、手順2で作ったアカウントでログイン
- Copilot の契約がないアカウントは、自動的に **Copilot Free**（無料プラン）に登録される
  - クレジットカードの登録などは一切不要




画像出典:Visual Studio Code Docs(CC BY 3.0 US)

# 4-2. ブラウザで認可する

- **認可を実行**:
  - VS Code からのアクセス許可画面が出たら、緑の **Authorize** ボタンを押す
  - （右は画面の例。実際はアプリ名が Visual Studio Code と表示されます）
- **エディタに戻る**:
  - 「Visual Studio Code を開きますか?」と聞かれたら許可して戻る
- **完了確認**:
  - 右下の Copilot アイコンを開き、プラン名（Copilot Free など）が表示されていれば完了！




画像出典:GitHub Docs(CC BY 4.0)

## 5. Markdown で補完を有効にする

- 今回はプログラムではなく **文章** を書くので、Markdown ファイル ( `.md` ) を使う
- Copilot の補完は、初期設定では **Markdown やプレーンテキストでは無効** になっている場合がある
- **有効にする方法**:
  i. `.md` ファイルを開いた状態で、右下ステータスバーの Copilot アイコンをクリック
  ii. メニューから **Markdown** での補完を有効にする
- **設定ファイルで指定する場合** ( `settings.json` ):

```
"github.copilot.enable": {
  "*": true,
  "markdown": true,
  "plaintext": true
}
```

## データの扱いとプライバシー設定

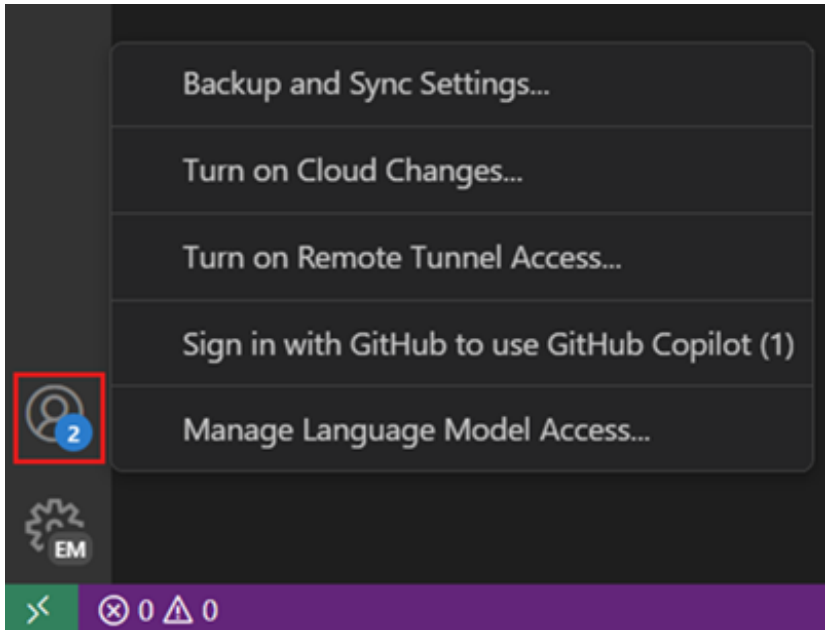
文章を書き始める前に、初期設定と変更方法を把握しておきましょう。

- **Copilot Free の初期設定**:
  - テレメトリ（製品改善のためのデータ送信）: **有効**
  - 公開コードと一致する提案（Public code suggestions）: **許可**
- **より厳格に変更したい場合**:
  - **VS Code 設定**: `telemetry.telemetryLevel` を検索して `off` に設定
  - **GitHub 設定**: 🔗 https://github.com/settings/copilot を開き、
    - 「Suggestions matching public code」を **Block** に変更
    - 「Allow GitHub to use my data for AI model training」を **Disabled** に変更

> ⚠️ 個人情報や、未発表の大切な原稿は入力しないように注意

## トラブルシューティング（うまくいかないとき）

- **Sign In ボタンや Copilot アイコンが見当たらない**
  - 左下の人型アイコン（Accounts）→ **Sign in with GitHub to use GitHub Copilot**
  - VS Code のバージョンを確認（1.116 以降が必要。古ければアップデート）
- **別のアカウントでサインインしてしまった**
  - Accounts メニューから **Sign out** して正しいアカウントで入り直す
- **文章の補完が出ない**
  - ファイルを `.md` 付きで保存しているか確認
  - Markdown での補完が有効になっているか確認（手順 5）



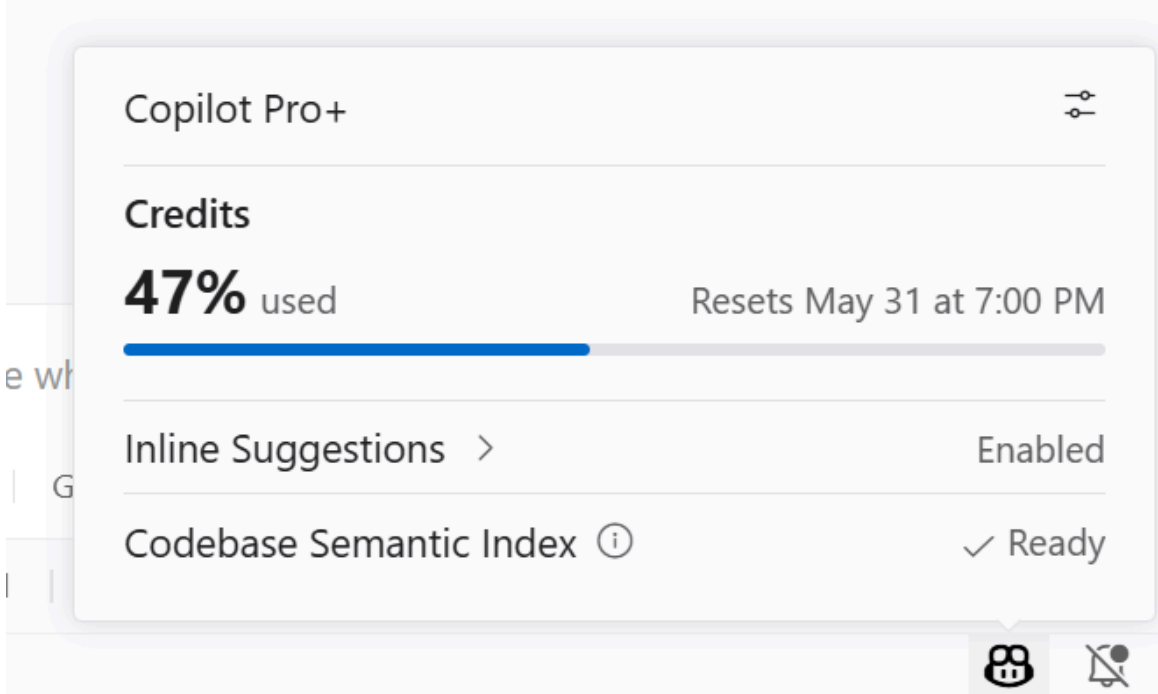

画像出典:Visual Studio Code Docs(CC BY 3.0 US)

## セットアップが早く終わったら

- **使用量を確認する**
  - 今月の利用枠がどれくらい残っているかをチェック
- **GitHub Student Developer Pack を申請する**
  - 認証されれば、上限を気にせず使える (反映に数日かかることも)
- **まわりの人を手伝う**
  - 困っている人がいたら声をかけて

# 使用量を確認する

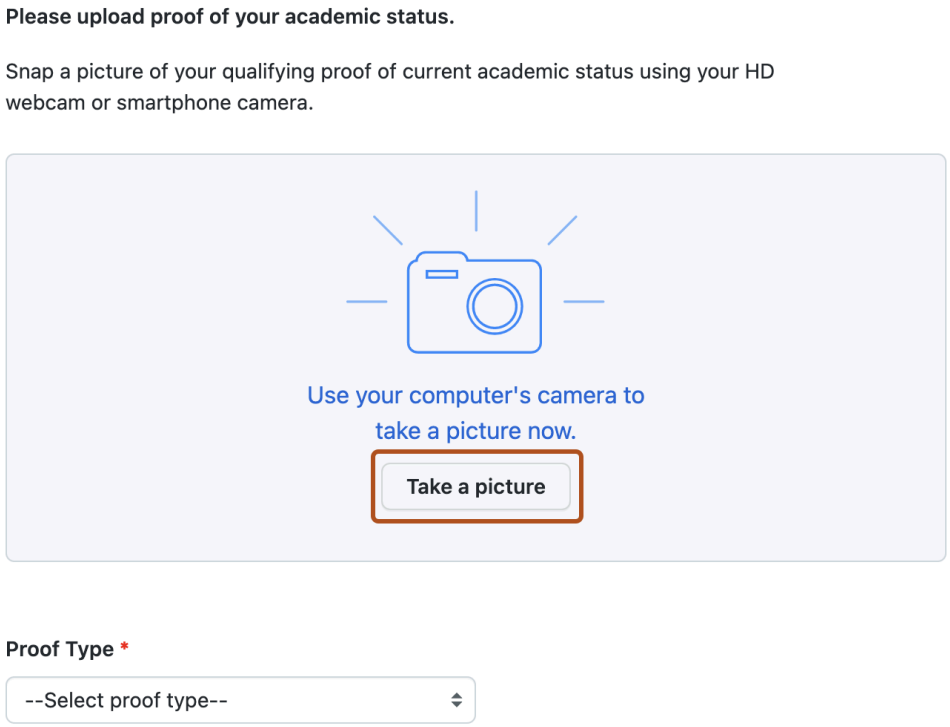
- 右下ステータスバーの Copilot アイコンをクリック
- 今月の利用枠を何%消費したかが表示される
  - （右は有料プランの例。Free ではコード補完の使用量も表示）
- **Copilot Free の月間上限**:
  - **補完**: 2,000回まで（確定ベース）
  - **チャット**: AIクレジットの範囲内（月50回程度が目安）
- 実験に入る前に、残り枠を確認しておきましょう




画像出典:Visual Studio Code Docs(CC BY 3.0 US)

# 上限を気にせず使うには？

- **GitHub Student Developer Pack**:
  - 学生認証を行うと、有料相当の **Copilot Student** を **無料** で利用できます
- **申請・有効化の手順**:
  i. https://github.com/settings/education/benefits にアクセス
  ii. 大学のメールアカウント (ac.jp) を登録し、学生証の写真等を提出して申請
  iii. 認証完了後、同ページで Copilot Student を有効化
  - ※ 認証反映に数日かかる場合があります
- 詳細: GitHub Docs「Copilot Student の設定」

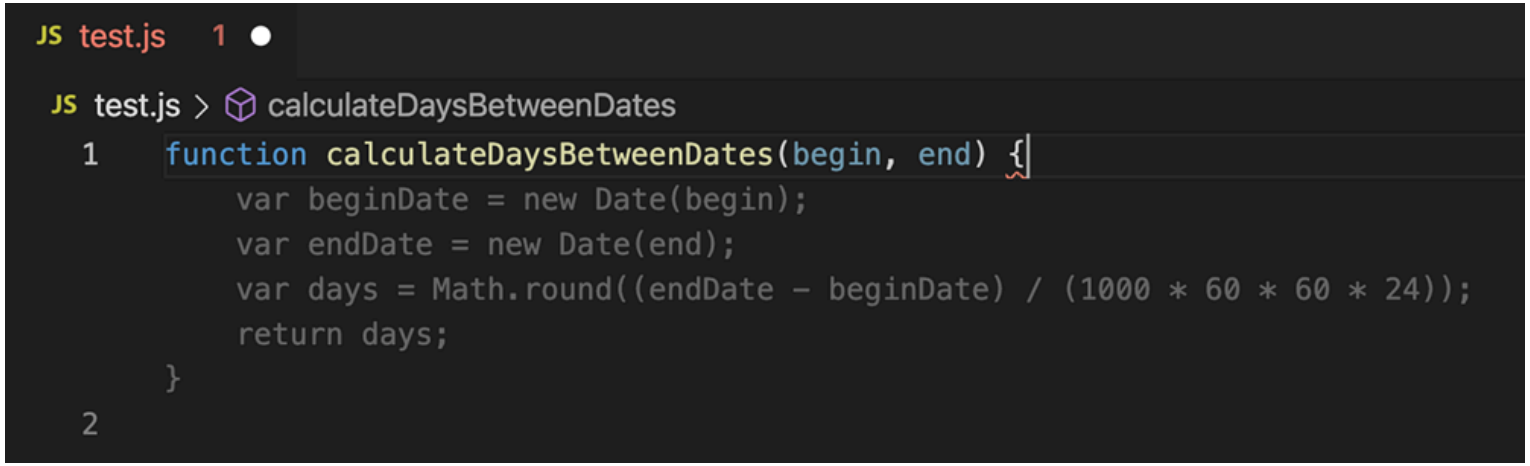



画像出典:GitHub Docs(CC BY 4.0)

# 補完の基本操作

# 6. テキスト補完を試す

- `File > New Text File` で新規ファイルを作り、`story.md` などの名前で保存
- 文章を書き始めると、続きの文章が **薄いグレー（ゴーストテキスト）** で提案される




- （図はプログラムの例。文章でも同じようにグレーの文字で続きが提案される）
- Tab: **提案を確定** / Esc: **提案を破棄**
- 👉 **試してみよう**: `# 雨の日の図書館` と見出しを書き、1行目を書き始めて少し待つ

画像出典:Visual Studio Code Docs(CC BY 3.0 US)

## 補完をコントロールする

- 提案を **全部は受け入れない** ことも、共創の大切な判断
- 便利な操作
  - **単語ごとに確定**: Windows `Ctrl` + `→` / macOS `⌘ Command` + `→`
    - 気に入った部分までだけ受け入れ、その先は自分で書く
  - **別の候補を見る**: Windows `Alt` + `]` / macOS `⌥ Option` + `]`
    - 提案にマウスを重ねると、候補を切り替えるツールバーも表示される
- グレーの提案は、Copilot が予測した **「確率の高い続き」**
  - 候補は一つとは限らない (前回の Transformer Explainer で見た候補のリスト)
- 提案は、直前までの文章 (= プロンプト) によって変わる
  - 書き出しの言葉、文体、見出しを変えると、続きの方向も変わる

# 4つの実験

# 4つの実験

補完を使って、前回の講義で考えたことを自分の手で確かめる

| 実験 | やること | 確かめること |
|---|---|---|
| **1. 平均を観察する** | 全員が同じ書き出しで書き、提案を比べる | 平均への引力 |
| **2. 文脈を設計する** | 書き出しの上に設定メモを書き足す | ボタンの手前 |
| **3. 平均から外れる** | 別の候補を選ぶ、1語だけ自分で変える | 向こうから次の手前へ |
| **4. 疑う** | 参考文献を補完させ、実在を確かめる | AIの誤り |

## 実験1: 平均を観察する

- `yuuhi.md` という名前で新しいファイルを作り (全員同じ名前)、次の2行を入力して待つ

```
# 夕日

窓の外の夕日は
```

- 提案の最初の一文を確定し、**Mentimeter に投稿**
- 👉 全員の提案は、どれくらい似通う?
  - 似ていれば → 「典型的な夕日」の言語版
  - ばらつけば → 生成に含まれる偶然 (前回の「シード値というサイコロ」)

# 実験2: 文脈を設計する

- 書き出しの上に「設定メモ」を書き足して、提案を比べる

```
# 夕日

- 語り手: 100年後の火星で暮らす子ども
- 文体: 短い文、ひらがなを多めに

窓の外の夕日は
```

- 火星の夕日は **青く見える** ことが知られている
  - AIは設定メモに引っぱられる? それとも橙色の典型的な夕日に戻る?
- 語り手、時代、場所、文体、見出しを自由に変えて試す

火星の夕日 (NASA)

## 補完は「指示」ではなく「続き」

- Copilot の補完は、チャットのように指示に従っているわけではない
  - ファイルに書かれた文章を **文書の一部** として読み、その続きとしてありそうな言葉を予測する
- つまり、**ファイルに書かれている文章そのものがプロンプト**
- 何をどこまで書いておくかを決めること
  - = 補完における **ボタンの手前** の仕事

## 実験3: 平均から外れる

- **別の候補から選ぶ**
  - Alt (⌥) + ] で候補を切り替え、いちばん変だと思うものを選ぶ
- **1語だけ自分で変える**
  - Ctrl (⌘) + → で途中まで受け入れ、続く1語を思いがけない言葉に書き換える
  - 例: 「窓の外の夕日は、」のあとに「**冷蔵庫の**」と書き足してから、提案を待つ
- 👉 1語の介入で、その先の提案がすべて変わる
  - 前回: 「向こうで受け取ったものは、次の手前を変えていく」
  - 補完では、選んだ言葉・書き足した言葉が、すぐに次のプロンプトになる

## 実験4: 疑う

- ファイルの末尾に次のように書いて、提案を待つ → どんな文献が提案される?

```
## 参考文献

1.
```

- 2〜3件確定したら、実在するかを検索で確かめる
  - Google Scholar / CiNii Research / 国立国会図書館サーチ
- 言語モデルは、存在しない文献を、もっともらしい体裁で提案することがある
  - 実在する文献が出ることもあるが、確かめなければ見分けがつかない

> ⚠️ レポートで参考文献を書くときは、必ず実在を確かめる

## 発展: 頼む (チャット) と続ける (補完) を比べる

- 時間が余った人は、ふだん使っているチャットAI (ChatGPT、Gemini など) に頼んでみる
  - 「『窓の外の夕日は』に続く文章を書いてください」
- 補完の提案と比べてみる
  - 前置きは付いている? 文章はどれくらい整っている?
  - 前回の **「人間の好みに合わせる調整」** は、文章の手触りにどう表れている?
- Copilot Free のチャットは回数に上限があるので、ふだん使っているチャットAIでOK

# 次週までの課題
# AIと共創するレポート
# 「創作において人工知能とは…」

# 課題: AIと共創するレポート

- VS Code と GitHub Copilot の **テキスト補完** を使って、レポートを書く
- 書き出しは必ず次の一文から始める

> **# 創作において人工知能とは**
>
> 創作において人工知能とは、

- この続きを、AIの補完と自分の言葉を往復させながら書き進める
- 分量の目安: **1,200〜2,000字程度**
- AIの提案をそのまま並べるのではなく、**採用・修正・拒否の判断** を重ねて、自分のレポートに仕上げる

## 進め方

1. `report.md` という名前で新しいファイルを作り、書き出しの一文を入力する
2. 少し待って、グレーの文字で提案される続きを読む
3. 提案を **全部確定する / 単語ごとに確定する / 破棄して自分で書く** を選ぶ
    - Tab: 確定 / Ctrl (⌘) + →: 単語ごとに確定 / Esc: 破棄
4. 方向を変えたいときは、自分で数語を書き足してから、再び提案を待つ
    - 書き出しの言葉や文体を変えると、続きの方向も変わる
5. 前回と今回の内容 (ボタンの手前・奥・向こう、平均への引力、今日の実験など) も手がかりにする
6. 参考文献を書くときは、実在するかを必ず確かめる

## 共創メモ

- レポートの最後に「`## 共創メモ`」という見出しを付けて、AIとの共創をふりかえる
  - 200〜400字程度 (レポートの字数には含めない)
- 書くことの例
  - 印象に残ったAIの提案と、それを受け入れた (または書き換えた) 理由
  - 断った提案と、断った理由
  - 方向を変えるために、自分で書き足したり工夫したりしたこと

## 提出方法

- **提出物**: テキストファイル ( `report.md` )
  - レポート本文と共創メモを、1つのファイルにまとめる
  - ファイル名は `学籍番号_氏名.md` に変更して提出
- **提出先**: (提出先を記入)
- **締切**: 次回の授業の前日まで
- 注意
  - 書き出しの一文「創作において人工知能とは、」から始めること
  - 個人情報や、他人に見せたくない内容は入力しないこと

## 書き始めよう

- 授業の残りの時間で `report.md` を作り、書き始める
  - 書き出しの一文と、最初の段落まで
- 補完がうまく表示されない人は、この時間に質問を

# まとめ

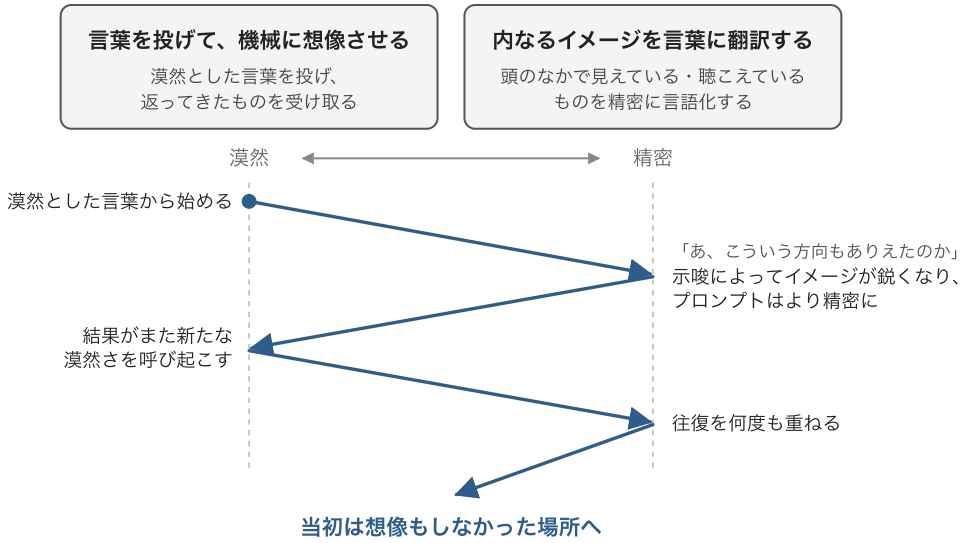
## 4つの実験で確かめたこと

- 全員が同じ書き出しで書くと、提案は似通いやすい
  - → **平均への引力**
- 書き出しの前に何を書いておくかで、提案の方向が変わる
  - → **ボタンの手前**
- 1語を選び直すだけで、その先のすべてが変わる
  - → **向こうで受け取ったものが、次の手前を変える**
- AIは、もっともらしい誤りを提案することがある
  - → **確かめるのは人間の仕事**

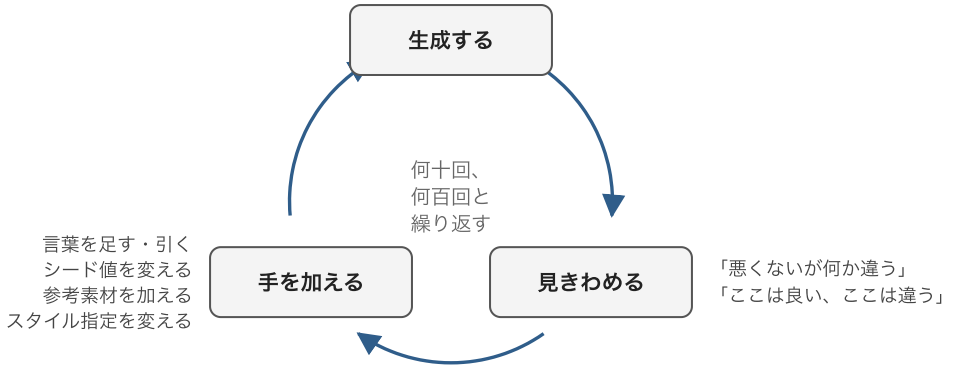
# 提案を選ぶ

- (a) 漠然とした言葉から始め、AIの返答に気づかされながら、イメージを鋭くしていく
- (b) 生成する → 見きわめる → 手を加える
  - 画像生成では、何十回、何百回と繰り返す
  - 補完では、このサイクルが **数秒ごとに回る**
- **使いこなす = 提案を受け入れる・書き換える・断るを、自分で選べること**
- 👉 Mentimeter:「AIの提案を断った瞬間」をひとこと

(a) 二つの態度と、そのあいだの往復




(b) 実践のサイクル



## 参考文献


- Botnik Studios. “Harry Potter and the Portrait of What Looked Like a Large Pile of Ash.” 2017.
- NASA. “Sunset at Mars’ Gale Crater (Curiosity).” 2015.
- Visual Studio Code Docs. “Inline suggestions from GitHub Copilot in VS Code.”
- 京都産業大学. 生成AI利用ガイドライン, 2026.